

ATCD MODULE 1

1 a) What is automata theory and explain alphabets, strings, languages, with examples?

Automata theory, also known as the Theory of Computation, is a field within computer science and mathematics that focuses on studying abstract machines to understand the capabilities and limitations of computation by analyzing mathematical models of how machines can perform calculations.

Basics of string:

Alphabet

- Definition: An alphabet is any finite set of symbols.
- Example: $\Sigma = \{a, b, c, d\}$ is an alphabet set where 'a', 'b', 'c', and 'd' are symbols.

String

- Definition: A string is a finite sequence of symbols taken from Σ .
- Example: 'cabcad' is a valid string on the alphabet set $\Sigma = \{a, b, c, d\}$

Length of a String

- Definition: It is the number of symbols present in a string. (Denoted by $|S|$).
- Example: If $S = \text{'cabcad'}$, $|S| = 6$

Prefix of a string

- Definition: It is the string formed by taking any number of symbols of the string.
- Example: String $w = 0111$

Prefix of a string for 0111 $\rightarrow \lambda, 0, 01, 011, 0111$

Substring

- Definition: It is the string formed by sequence of consecutive symbols taken from string.
- Example: String $w = 0111$

Substring of a string for 0111 $\rightarrow 0, 1, 01, 11, 011, 111$

Proper Prefix of a string

- Definition: Any Prefix of the string other than the string itself.
- Example: String $w = 0111$

Proper Prefix of a string for 0111 $\rightarrow \lambda, 0, 01, 011$

Suffix of a string

- Definition: It is the string formed by taking any number of symbols from the end of the string.
- Example: String $w = 0110$

Suffix of a string for 0111 $\rightarrow \lambda, 0, 10, 110, 0110$.

Proper Suffix of a string

- Definition: Any Suffix of the string other than the string itself.
- Example: String $w = 0110$

Proper Suffix of a string for 0111 $\rightarrow \lambda, 0, 10, 110$.

Language

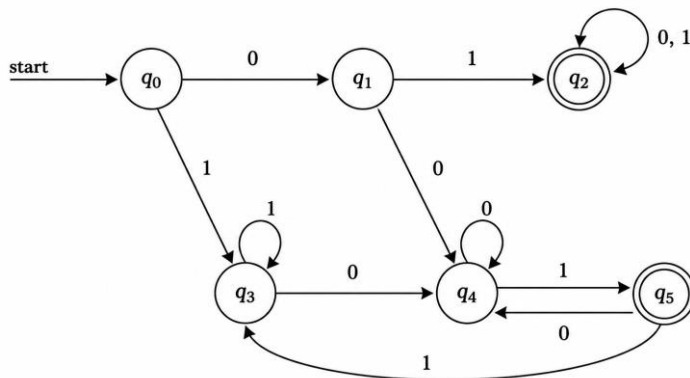
- Definition: A language is a subset of Σ^* for some alphabet Σ . It can be finite or infinite.
- Example: If the language takes all possible strings of length 2 over $\Sigma = \{a, b\}$, then $L = \{ab, bb, ba, aa\}$
- Operations on Languages:
 - Union
 - Concatenation
 - Intersection

1 b) Design a FA over $\Sigma = \{0,1\}$ that accepts strings which either start with 01 or end with 01?

The string starts with 01, or the string ends with 01.

$L = \{w \in \{0,1\}^* \mid w \text{ starts with } 01 \text{ or ends with } 01\}$

DFA that accepts all strings that either begin with 01 or finish with 01 over $\Sigma = \{0,1\}$



Step 1: Construct the states:

$Q = \{q_0, q_1, q_2, q_3, q_4, q_5\}$

- $q_0 \rightarrow$ Initial state.
- $q_1 \rightarrow$ First symbol is 0; waiting for 1.
- $q_2 \rightarrow$ String started with 01.
- $q_3, q_4, q_5 \rightarrow$ Used to keep track of whether the string ends in 01.
- $q_5 \rightarrow$ String currently ends with 01, so it is a **final state**.

Step 2: Transitions for strings starting with 01

From q_0 : $q_0 \xrightarrow{0} q_1$

If the next symbol is 1: $q_1 \xrightarrow{1} q_2$

Therefore, the string started with 01.

Since the condition is already satisfied, q_2 loops on both inputs: $q_2 \xrightarrow{0,1} q_2$

So q_2 is an accepting state.

If q_1 receives another 0, the string does not start with 01, so: $q_1 \xrightarrow{0} q_4$

Step 3: Track strings ending with 01

For strings that do not start with 01, we track their last symbols.

From q_0 , if the first symbol is 1: $q_0 \xrightarrow{1} q_3$

At q_3 :

- On 1, remain at q_3 : $q_3 \xrightarrow{1} q_3$
- On 0, move to q_4 : $q_3 \xrightarrow{0} q_4$

At q_4 :

- On 0, remain at q_4 : $q_4 \xrightarrow{0} q_4$
- On 1, we have obtained 01 at the end: $q_4 \xrightarrow{1} q_5$

Therefore, q_5 is an accepting state.

From q_5 : $q_5 \xrightarrow{0} q_4$ because the latest symbol becomes 0, and $q_5 \xrightarrow{1} q_3$ because the ending is no longer 01.

Step 4: Transition Table

Present State	Input 0	Input 1
$\rightarrow (q_0)$	(q_1)	(q_3)
(q_1)	(q_4)	(q_2)
* (q_2)	(q_2)	(q_2)
(q_3)	(q_4)	(q_3)
(q_4)	(q_4)	(q_5)
* (q_5)	(q_4)	(q_3)

Step-5: Final DFA: Accepting states $F = \{q_2, q_5\}$

2 a) What is DFA? Design a DFA that accepts strings ending with 00.

Finite automata:

A finite automaton is a mathematical model of a system with discrete inputs and outputs.

Finite Automata can be classified into two types –

- Deterministic Finite Automata (DFA)
- Non-deterministic Finite Automata (NFA)

Deterministic finite automata:

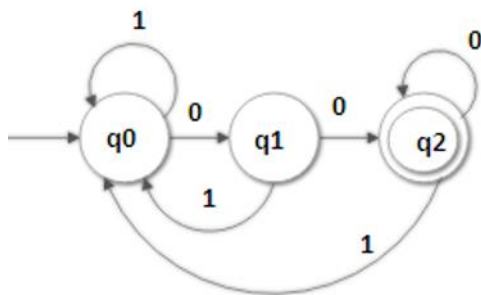
Deterministic Finite Automaton is a FA in which there is only one path for a specific input from current state to next state.

Tuples: $(Q, \Sigma, \delta, q_0, F)$

- Q is a finite set of states.
- Σ is a finite set of input alphabets
- δ is the transition function. $\delta: Q * \Sigma \rightarrow Q$
- q_0 is the initial state
- F is a set of final state/states of Q

DFA that accepts all strings over $\Sigma=\{0,1\}$ that end with 00:

$$L=\{w \in \{0,1\}^* \mid w \text{ ends with } 00\}$$



Step-1: Construct the states

$Q = \{q_0, q_1, q_2\}$

q_0 --> Initial State

q_1 --> String currently ends in one 0.

q_2 --> String ends with 00. (Final State)

Step-2: Transitions for ending with 00

From q_0

- On 0 $\rightarrow q_1$, because we have seen one 0.
- On 1 $\rightarrow q_0$, because the string does not end in 0.

$$\delta(q_0, 0) = q_1 \quad \delta(q_0, 1) = q_0$$

From q_1

- On 0 $\rightarrow q_2$, because now the last two symbols are 00.
- On 1 $\rightarrow q_0$, because the ending becomes 01, so it no longer ends in 0.

$$\delta(q_1, 0) = q_2 \quad \delta(q_1, 1) = q_0$$

From q_2

We are already ending in 00.

- On 0 $\rightarrow q_2$, because 000 still ends in 00.
- On 1 $\rightarrow q_0$, because after adding 1, the string no longer ends in 00.

$$\delta(q_2, 0) = q_2 \quad \delta(q_2, 1) = q_0$$

Step 3: Transition Table

Present State	Input 0	Input 1
$\rightarrow (q_0)$	(q1)	(q0)
(q1)	(q2)	(q0)
★ (q2)	(q2)	(q0)

Step-4: Formal DFA Representation

$$M = (Q, \Sigma, \delta, q_0, f)$$

$$Q = \{q_0, q_1, q_2\}$$

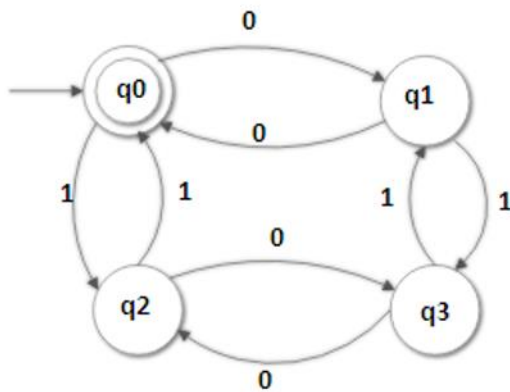
$$\Sigma = \{0, 1\}$$

$q_0 \rightarrow$ start state

$$f = \{q_2\}$$

2 b) Design DFA for even number of 0's and even number of 1's.

$$L = \{w \in \{0, 1\}^* \mid w \text{ contains an even number of 0's and an even number of 1's}\}$$



Step 1: Understand the requirement

- Whether the number of 0s is even or odd.
- Whether the number of 1s is even or odd.

0's	1's	State
Even	Even	(q0)
Odd	Even	(q1)
Even	Odd	(q2)
Odd	Odd	(q3)

Step 2: Explain each state

q0 : Even 0's, Even 1's

- This is the initial state.
- It is also the final state, because we want both counts to be even.

q1 : Odd 0's, Even 1's

- When we read a 0 from q0 , the number of 0's changes from even to odd.

q0 ---> 0 ---> q1

q2 : Even 0's, Odd 1's

- When we read a 1 from q0 , the number of 1's changes from even to odd.

q0 ---> 1 ---> q2

q3 : Odd 0's, Odd 1's

- When both counts become odd, we reach q3 .

Step 3: Construct the transitions

From q0

q0 has even 0's and even 1's.

- Reading 0 makes 0's odd: $q0 \xrightarrow{0} q1$
- Reading 1 makes 1's odd: $q0 \xrightarrow{1} q2$

From q1

q1 has odd 0's and even 1's.

- Reading another 0 makes 0's even: $q1 \xrightarrow{0} q0$
- Reading 1 makes 1's odd: $q1 \xrightarrow{1} q3$

From q2

q2 has even 0's and odd 1's.

- Reading 0 makes 0's odd: $q2 \xrightarrow{0} q3$
- Reading another 1 makes 1's even: $q2 \xrightarrow{1} q0$

From q3

q3 has odd 0's and odd 1's.

- Reading 0 makes 0's even: $q3 \xrightarrow{0} q2$
- Reading 1 makes 1's even: $q3 \xrightarrow{1} q1$

Step 4: Transition Table

Present State	Input 0	Input 1	State Meaning
→ ★ (q0)	(q1)	(q2)	Even 0's, Even 1's
(q1)	(q0)	(q3)	Odd 0's, Even 1's
(q2)	(q3)	(q0)	Even 0's, Odd 1's
(q3)	(q2)	(q1)	Odd 0's, Odd 1's

Step 5: Formal Representation

$M=(Q,\Sigma,\delta,q_0,F)$

$Q=\{q_0,q_1,q_2,q_3\}$

Input alphabet $\Sigma=\{0,1\}$

Input States q_0

Final States $F=\{q_0\}$

3 a) Difference between DFA and NFA.

Finite automata:

A finite automaton is a mathematical model of a system with discrete inputs and outputs.

Finite Automata can be classified into two types –

- Deterministic Finite Automata (DFA)
- Non-deterministic Finite Automata (NFA)

Deterministic finite automata:

Deterministic Finite Automaton is a FA in which there is only one path for a specific input from current state to next state.

Tuples: $(Q, \Sigma, \delta, q_0, F)$

- Q is a finite set of states.
- Σ is a finite set of input alphabets
- δ is the transition function. $\delta: Q * \Sigma \rightarrow Q$
- q_0 is the initial state
- F is a set of final state/states of Q

Non-deterministic finite automata:

NFA or Non Deterministic Finite Automaton is the one in which there exists many paths for a specific input from current state to next state.

Tuples: $(Q, \Sigma, \delta, q_0, F)$

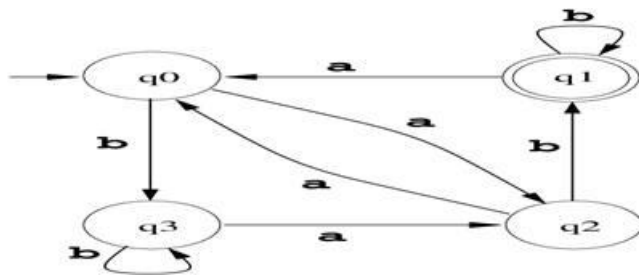
- Q is a finite set of states.
- Σ is a finite set of input alphabets
- δ is the transition function. $\delta: Q * \Sigma \rightarrow 2^Q$
- q_0 is the initial state
- F is a set of final state/states of Q

Difference between DFA and NFA

DFA	NFA
1) Deterministic Finite Automaton is a FA in which there is only one path for a specific input from current state to next state. Tuples: $(Q, \Sigma, \delta, q_0, F)$ $\delta: Q * \Sigma \rightarrow Q$	1) NFA or Non Deterministic Finite Automaton is the one in which there exists many paths for a specific input from current state to next state. Tuples: $(Q, \Sigma, \delta, q_0, F)$ $\delta: Q * \Sigma \rightarrow 2^Q$
2) DFA is difficult to construct. There is a unique transition on each input symbol.	2) NFA can be used in theory of computation because they are more flexible and easier to use than DFA.

3) Backtracking is allowed in DFA	3) In NFA, backtracking is not always possible.
4) Requires more space.	4) Requires less space.

3b) Check whether the strings (abbbabab, bbaaba)are accepted by the given state diagram or not.



The given diagram is an **NFA** because from q0 , on input a, there are **two possible transitions**:

q0 ---> a ---> q1 and q0 ---> a ---> q2

The **initial state** is q0, and the **final state** is q1 (double circle).

Transitions from the diagram

Q0 ---> a ---> {q1,q2}	q2 ---> a ---> q0
Q0 ---> b ---> q3	q2 ---> b ---> q1
Q1 ---> a ---> q0	q3 ---> a ---> q2
Q1 ---> b ---> q1	q3 ---> b ---> q3

Step-1. Check abbbabab:

Input	Current state(s)
Start	({q_0})
a	({q_1,q_2})
b	({q_1})
b	({q_1})
b	({q_1})
a	({q_0})
b	({q_3})
a	({q_2})
b	({q_1})

q0--- a--->{q1 ,q2 }--->b--->q1 --->b --->q1--->b--->q1--->a--->q0--->b--->q3--->a--->q2--->b--->q1

The string ends in **q1**, which is a final state.

abbbabab is ACCEPTED

Step-2. Check bbaaba

Input	Current state
Start	(q ₀)
b	(q ₃)
b	(q ₃)
a	(q ₂)
a	(q ₀)
b	(q ₃)
a	(q ₂)
q ₀ --->b--->q ₃ --->b--->q ₃ --->a--->q ₂ --->a--->q ₀ --->b--->q ₃ --->a--->q ₂	

The string ends in **q2**, which is **not** a final state.

bbaaba is REJECTED

Step-3. Final Answer

String	Final state	Result
abbbabb	(q ₁)	Accepted ✓
bbaaba	(q ₂)	Rejected ✗

5a) Define Regular Expression? Explain the Closure properties of Regular Expressions.

Regular Expression:

- The languages accepted by finite automata can be represented using simple patterns called **regular expressions**.
- A **regular expression (RE)** is a sequence of symbols used to describe a set of strings in a language.
- Regular expressions provide an easy and effective way to represent and analyze languages.
- Any language described by a regular expression is called a **regular language**.
- Any terminal symbol (i.e. an element of Σ), ϵ and ϕ are regular expressions.
- The union of two regular expressions R_1 and R_2 , written as $R_1 + R_2$, is also a regular expression.
- The concatenation of two regular expressions R_1 and R_2 , written as R_1R_2 , is also a regular expression.

- The iteration (or Closure) of a regular expression R, written as R^* is also a regular expression.
- If R is a regular expression, then (R) is also a regular expression.

Example

Regular

Expression:

a^*

Language: $\{ \epsilon, a, aa, aaa, aaaa, \dots \}$

5b) Construct a finite automaton that accepts a string w, where w is a binary number divisible by 3.

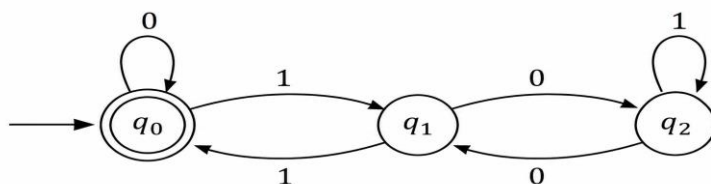
Step-1 : Possible Remainders

There are only three possible remainders: 0, 1, 2

So we need **three states**.

State	Meaning
q0	Remainder 0
q1	Remainder 1
q2	Remainder 2

Since a number is divisible by 3 when its remainder is **0**, q0 is the **final state**.



Step 3: Find the transitions

When a binary digit b is added to the right, the new value is: $2r+b$

where r is the previous remainder.

We calculate:

$\text{New remainder} = (2r+b) \bmod 3$

From q0 (remainder 0)

For input 0: $(2(0)+0)\text{mod}3=0$

So: $q_0 \xrightarrow{0} q_0$

For input 1: $(2(0)+1)\text{mod}3=1$

So: $q_0 \xrightarrow{1} q_1$

From q1 (remainder 1)

For input 0: $(2(1)+0)\text{mod}3=2$

So: $q_1 \xrightarrow{0} q_2$

For input 1: $(2(1)+1)\text{mod}3=0$

So: $q_1 \xrightarrow{1} q_0$

From q2 (remainder 2)

For input 0: $(2(2)+0)\text{mod}3=1$

So: $q_2 \xrightarrow{0} q_1$

For input 1: $(2(2)+1)\text{mod}3=2$

So: $q_2 \xrightarrow{1} q_2$

Step 3: Transition Table

Present State	Input 0	Input 1	Meaning
$\rightarrow \star (q_0)$	(q_0)	(q_1)	Remainder 0
(q_1)	(q_2)	(q_0)	Remainder 1
(q_2)	(q_1)	(q_2)	Remainder 2

Step-4 : Final Answer

The DFA is: $M=(Q,\Sigma,\delta,q_0,F)$

where

$Q=\{q_0, q_1, q_2\}$

$\Sigma=\{0,1\}$

q_0 =initial state

$F=\{q_0\}$

δ	0	1
q0	q0	q1
q1	q2	q0
q2	q1	Q2

6) Write about minimization of finite automata and explain with diagram step by step.

- Minimization of Finite Automata is the process of converting a given DFA into an equivalent DFA having the minimum possible number of states.
- The minimised DFA accepts the same language as the original DFA, but redundant or equivalent states are removed.
- Let a DFA be represented as: $M=(Q,\Sigma,\delta,q_0,F)$
where:
 - Q = finite set of states
 - Σ = input alphabet
 - δ = transition function
 - q_0 = initial state
 - F = set of final states
- **Equivalent States**
 - Two states p and q are equivalent if, for every possible input string w ,
 - $\delta^*(p,w) \in F \iff \delta^*(q,w) \in F$
- **Distinguishable States**
 - Two states are distinguishable if there exists at least one input string w such that:
 - $\delta^*(p,w) \in F$ but $\delta^*(q,w) \notin F$

Steps for DFA Minimization

Step 1: Remove unreachable states

- Starting from the initial state q_0 , find all states that can be reached.
- Any state that cannot be reached from q_0 is called an unreachable state and is removed.

Step 2: Initial Partition

- Divide all remaining states into two groups: $P_0 = \{F, Q-F\}$
Group 1 $\rightarrow$ Final states
Group 2 $\rightarrow$ Non-final states
- A final state and a non-final state can never be equivalent.

Step 3: Refine the Partitions

- If two states have transitions to different groups, they are distinguishable and must be separated.
- For example, if states p and q are in the same group but: $\delta(p,a) \in G_1$ and $\delta(q,a) \in G_2$ where $G_1 \neq G_2$, then p and q must be placed in different groups.

Step 4: Repeat Partitioning

- Continue the partition refinement process until no group can be divided further.
- When: $P_i = P_{i+1}$ the partitioning process is complete.
- The states present in the same final group are equivalent states.

Step 5: Construct the Minimized DFA

- Each equivalence class is represented by one state.
- If the final partition is: $\{q_0, q_2\}, \{q_1\}, \{q_3, q_4\}$ then the minimized DFA contains three states: $[q_0, q_2], [q_1], [q_3, q_4]$

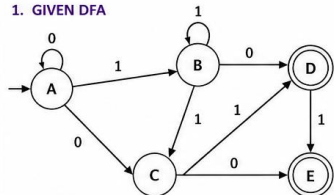
Need of Minimization:

- Reduces the number of states.
- Reduces memory and implementation requirements.

EXAMPLE: MINIMISATION OF DFA

Minimise the following DFA over $\Sigma = \{0, 1\}$.

1. GIVEN DFA



State	$\delta(\text{state}, 0)$	$\delta(\text{state}, 1)$
A	A	B
B	D	C
C	E	D
D	D	E
E	E	E

$Q = \{A, B, C, D, E\}$, $\Sigma = \{0, 1\}$, $q_0 = A$, $F = \{D, E\}$

2. REMOVE UNREACHABLE STATES

All the states are reachable from A. So, no state is removed.

3. INITIAL PARTITION

Final states = $\{D, E\}$

Non-final states = $\{A, B, C\}$

$P_0 = \{\{D, E\}, \{A, B, C\}\}$

4. REFINING THE PARTITIONS

$P_0 = \{\{D, E\}, \{A, B, C\}\}$

State	0	1	Group
A	A	B	$\{A, B, C\}$
B	D	C	$\{D, E\}, \{A, B, C\}$
C	E	D	$\{D, E\}, \{D, E\}$

In $\{A, B, C\}$:

- $A \rightarrow (0 \text{ to } \{A, B, C\}, 1 \text{ to } \{A, B, C\})$
- $B \rightarrow (0 \text{ to } \{D, E\}, 1 \text{ to } \{A, B, C\})$
- $C \rightarrow (0 \text{ to } \{D, E\}, 1 \text{ to } \{D, E\})$

A is different from B and C.

Also B and C are different.

So, $\{A, B, C\}$ is split into $\{A\}, \{B\}, \{C\}$.

$P_1 = \{\{D, E\}, \{A\}, \{B\}, \{C\}\}$

State	0	1	Group
A	A	B	$\{A\}$
B	D	C	$\{D, E\}, \{C\}$
C	E	D	$\{D, E\}, \{D, E\}$
D	D	E	$\{D, E\}, \{D, E\}$
E	E	E	$\{D, E\}, \{D, E\}$

In $\{D, E\}$:

- $D \rightarrow (0 \text{ to } \{D, E\}, 1 \text{ to } \{D, E\})$
- $E \rightarrow (0 \text{ to } \{D, E\}, 1 \text{ to } \{D, E\})$

Both D and E have same transitions.

So they are equivalent.

No more partitions possible.

Hence, $P_2 = P_1$ (stable partition).

5. EQUIVALENT STATES AND MINIMISED DFA

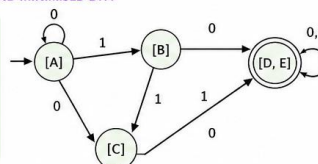
Equivalent states:

$D = E$

Other states are different.

Equivalence classes:

$\{A\}, \{B\}, \{C\}, \{D, E\}$



6. TRANSITION TABLE OF MINIMISED DFA

State	0	1
[A]	[A]	[B]
[B]	[D, E]	[C]
[C]	[D, E]	[D, E]
[D, E]	[D, E]	[D, E]

Initial state: [A]

Final state(s): [D, E]

Thus, the given DFA with 5 states is minimised to a DFA with 4 states while accepting the same language.

7) Convert the following regular grammars into finite automata.

i). $S \rightarrow aA/bB/a/b$

$A \rightarrow aS/bB/b$

$B \rightarrow aA/bS.$

ii). $S \rightarrow aA/bS$

$A \rightarrow bB/a$

$B \rightarrow aS/b.$

Rules:

- Each non-terminal becomes a state.
- Create one new state **F** as the final state.
- For a production $A \rightarrow aB$, draw: $A \xrightarrow{a} B$
- For a production $A \rightarrow a$, draw: $A \xrightarrow{a} F$

- The start symbol S becomes the **initial state**.
- F is the **final state**.

7. Convert the following Regular Grammars into Finite Automata.

Rule Used : For right linear grammar, the equivalent FA is constructed as

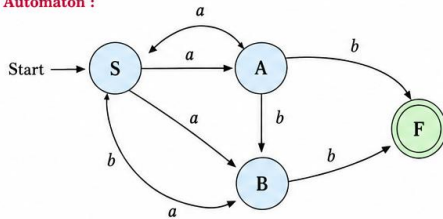
- Create one state for each non-terminal.
- For a production $A \rightarrow aB$, add transition $A \xrightarrow{a} B$.
- For a production $A \rightarrow a$, add transition $A \xrightarrow{a} F$.

i). $S \rightarrow aA / bB / a / b$
 $A \rightarrow aS / bB / b$
 $B \rightarrow aA / bS$

Explanation :

- > Non-terminals are S, A, B .
- > New final state is F .
- > For each production, add corresponding transitions.
- > For $S \rightarrow a, S \rightarrow b$ add transitions to final state F .

Finite Automaton :



Transition Table :

State	a	b
S	A, F	B, F
A	S	B, F
B	A	S
F	—	—

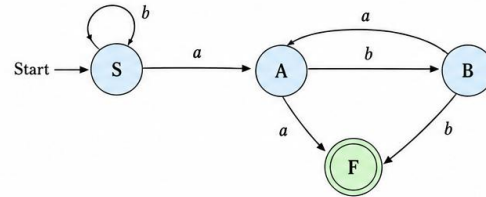
Start state : S
 Final state : F

ii). $S \rightarrow aA / bS$
 $A \rightarrow bB / a$
 $B \rightarrow aS / b$

Explanation :

- > Non-terminals are S, A, B .
- > New final state is F .
- > For each production, add corresponding transitions.
- > For $A \rightarrow a, B \rightarrow b$ add transitions to final state F .

Finite Automaton :



Transition Table :

State	a	b
S	A	S
A	F	B
B	S	F
F	—	—

Start state : S
 Final state : F

Hence, the above are the finite automata equivalent to the given regular grammars.

(i) Transitions

- $S \rightarrow aA \rightarrow Sa A$
- $S \rightarrow bB \rightarrow Sb B$
- $S \rightarrow a \rightarrow Sa F$
- $S \rightarrow b \rightarrow Sb F$
- $A \rightarrow aS \rightarrow Aa S$
- $A \rightarrow bB \rightarrow Ab B$
- $A \rightarrow b \rightarrow Ab F$
- $B \rightarrow aA \rightarrow Ba A$
- $B \rightarrow bS \rightarrow Bb S$

(ii) Transitions

- $S \rightarrow aA \rightarrow Sa A$

- $S \rightarrow bS \rightarrow Sb S$
- $A \rightarrow bB \rightarrow Ab B$
- $A \rightarrow a \rightarrow Aa F$
- $B \rightarrow aS \rightarrow Ba S$
- $B \rightarrow b \rightarrow Bb F$

8 a) Explain the pumping lemma for regular languages and its applications.

Pumping Lemma for Regular Languages:

- It is a necessary condition for a language to be regular.
- It helps us prove that a given language is not regular by showing that it violates the lemma.

Statement:

- If L is a regular language then there exists an integer p ($p \geq 1$) called pumping length such that any string w belongs to L with $|w| \geq p$ can be written as: $w = xyz$.

Where, $|y| > 0$ (y is not empty)

$|x| \leq p$ and for all $i \geq 0$, $x y^i z$ belongs to L .

Intuition:

- If a language is regular, it is accepted by some finite automation with a finite number of states.
- When a long string (length $\geq p$) is processed, some state must repeat.
- The part of string causing loop can be pumped and resulting string must still be in the language.

Applications:

- To prove that a language is not regular.
- Widely used in theory of computation for proofs.
- Helps in distinguishing between regular and non-regular languages.

8 b) Prove that the following languages are not regular by using pumping lemma.

$L = \{ a^n b^n \text{ where } n \geq 1 \}$ is not regular.

Proof that $L = \{a^n b^n \mid n \geq 1\}$ is not regular using Pumping Lemma

Step 1 Assume L is regular

Assume, for contradiction, that L is a regular language.

By Pumping Lemma, there exists a number $p \geq 1$ (pumping length) such that any string $w \in L$ with $|w| \geq p$ can be written as $w = xyz$ satisfying:

- 1 $|y| > 0$
- 2 $|xy| \leq p$
- 3 $xy^i z \in L$ for all $i \geq 0$

Step 2 Choose a suitable string

Choose $w = a^p b^p$

This string has p a 's followed by p b 's.

Clearly, $w \in L$ (number of a 's = number of b 's).

Also, $|w| = 2p \geq p$. So, Pumping Lemma applies to w .

Example: if $p = 4$

$w = a^p b^p = \underbrace{aaaa}_{p \text{ a's}} \underbrace{bbbb}_{p \text{ b's}}$

Step 3 Divide the string as $w = xyz$

According to the lemma, $w = xyz$ such that

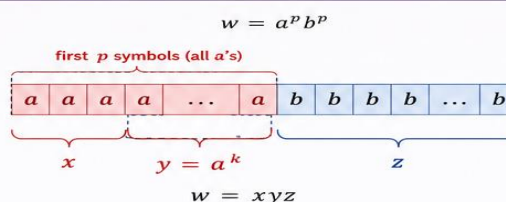
$|y| > 0$ and $|xy| \leq p$.

Since $|xy| \leq p$, both x and y must lie within the first p symbols of w .

The first p symbols of w are all a 's.

Therefore, y contains only a 's.

So, $y = a^k$ for some $k \geq 1$ (because $|y| > 0$).



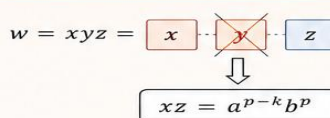
Step 4 Pump the substring y

By Pumping Lemma, $xy^i z \in L$ for all $i \geq 0$.

Choose $i = 0$.

Then $xy^0 z = xz$ (since $y^0 = \epsilon$, empty string).

We remove $y = a^k$ from w .



We removed $y = a^k$ from w .

Step 5 Compare the number of a 's and b 's

In $xz = a^{p-k} b^p$,

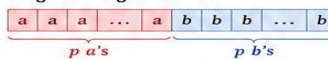
- Number of a 's = $p - k$
- Number of b 's = p

Since $k \geq 1$, $p - k \neq p$.

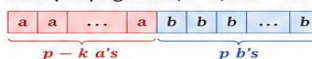
So, the number of a 's is not equal to the number of b 's.

Hence, $xz = a^{p-k} b^p \notin L$.

Original string $w = a^p b^p$



After pumping down ($i = 0$): $xz = a^{p-k} b^p$



Since $p - k \neq p$, $xz \notin L$.

Step 6 Obtain the contradiction

But Pumping Lemma requires $xy^i z \in L$ for every $i \geq 0$.

For $i = 0$, we got $xz \notin L$.

This contradicts the Pumping Lemma.

Contradiction!

Hence, our assumption that L is regular is false. ✗

Conclusion

Therefore, $L = \{a^n b^n \mid n \geq 1\}$ is **not regular**.

9) Define a regular grammar. Find the FA that recognizes the language generated by the following right linear grammar.

$S \rightarrow 0A/1B/1C$

$A \rightarrow 1A/0C$

$B \rightarrow 1B/0/1$

$C \rightarrow 0C/0/1A$

Regular Grammar:

- It is a type of grammar in which all productions are of the form $A \rightarrow aB$, $A \rightarrow a$. Where A, B are non-terminals, a is terminal.
- Every right linear grammar generates a regular languages and can be converted into a finite automation.

Conversion of Right Grammar into FA:

For a right linear grammar:

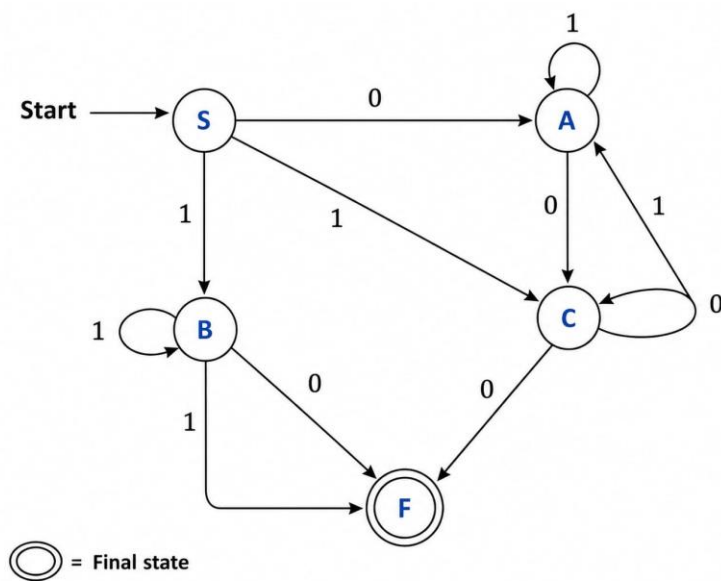
- Create one state for each non-terminal.
- If a production is $A \rightarrow aB$, add transitions from state A to state B on input a.
- If a production is $A \rightarrow a$, add transitions from state A to final state on input a.
- If $A \rightarrow \epsilon$, then make state A a final state.

Step-1: Given Right Grammar

- $S \rightarrow 0A/1B/1C$
- $A \rightarrow 1A/0C$
- $B \rightarrow 1B/0/1$
- $C \rightarrow 0C/0/1A$

Step-2: Identify States

- Non-terminals: S, A, B, C
- Create States: S, A, B, C
- Final state: F



Step-3: Add transitions using productions

$S \rightarrow 0A \Rightarrow S \xrightarrow{0} A$

$S \rightarrow 1B \Rightarrow S \xrightarrow{1} B$

$S \rightarrow 1C \Rightarrow S \xrightarrow{1} C$

$A \rightarrow 1A \Rightarrow A \xrightarrow{1} A$

$A \rightarrow 0C \Rightarrow A \xrightarrow{0} C$

$B \rightarrow 1B \Rightarrow B \xrightarrow{1} B$

$B \rightarrow 0 \Rightarrow B \xrightarrow{0} F$

$B \rightarrow 1 \Rightarrow B \xrightarrow{1} F$

$C \rightarrow 0C \Rightarrow C \xrightarrow{0} C$

$C \rightarrow 0 \Rightarrow C \xrightarrow{0} F$

$C \rightarrow 1A \Rightarrow C \xrightarrow{1} A$

Step-4: Transition Table

State	0	1
S	A	{B,C}
A	C	A
B	F	{B,F}
C	{C,F}	A
F	-	-

Final State: F

10 a) Write about equivalence of DFA and NFA?

- DFA (Deterministic Finite Automaton) and NFA (Nondeterministic Finite Automaton) are two types of finite automata used to recognize regular languages.
- Although their transition mechanisms are different, they have equal computational power.
- DFA and NFA are equivalent in power
- That means every language accepted by an NFA can also be accepted by a DFA, and every DFA is itself a special case of an NFA.

Deterministic finite automata:

Deterministic Finite Automaton is a FA in which there is only one path for a specific input from current state to next state.

Tuples: $(Q, \Sigma, \delta, q_0, F)$

- Q is a finite set of states.
- Σ is a finite set of input alphabets
- δ is the transition function. $\delta: Q * \Sigma \rightarrow Q$
- q_0 is the initial state
- F is a set of final state/states of Q

Non-deterministic finite automata:

NFA or Non Deterministic Finite Automaton is the one in which there exists many paths for a specific input from current state to next state.

Tuples: $(Q, \Sigma, \delta, q_0, F)$

- Q is a finite set of states.
- Σ is a finite set of input alphabets
- δ is the transition function. $\delta: Q * \Sigma \rightarrow 2^Q$
- q_0 is the initial state
- F is a set of final state/states of Q

10 b) Difference between DFA and NFA?

Deterministic finite automata:

Deterministic Finite Automaton is a FA in which there is only one path for a specific input from current state to next state.

Tuples: $(Q, \Sigma, \delta, q_0, F)$

- Q is a finite set of states.
- Σ is a finite set of input alphabets
- δ is the transition function. $\delta: Q * \Sigma \rightarrow Q$
- q_0 is the initial state
- F is a set of final state/states of Q

Non-deterministic finite automata:

NFA or Non Deterministic Finite Automaton is the one in which there exists many paths for a specific input from current state to next state.

Tuples: $(Q, \Sigma, \delta, q_0, F)$

- Q is a finite set of states.
- Σ is a finite set of input alphabets
- δ is the transition function. $\delta: Q * \Sigma \rightarrow 2^Q$
- q_0 is the initial state
- F is a set of final state/states of Q

DFA	NFA
1)Deterministic Finite Automaton is a FA in which there is only one path for a	1)NFA or Non Deterministic Finite Automaton is the one in which there

specific input from current state to next state. Tuples: $(Q, \Sigma, \delta, q_0, F)$ $\delta: Q * \Sigma \rightarrow Q$	exists many paths for a specific input from current state to next state. Tuples: $(Q, \Sigma, \delta, q_0, F)$ $\delta: Q * \Sigma \rightarrow 2^Q$
2) DFA is difficult to construct. There is a unique transition on each input symbol.	2) NFA can be used in theory of computation because they are more flexible and easier to use than DFA.
3) Backtracking is allowed in DFA	3) In NFA, backtracking is not always possible.
4) Requires more space.	4) Requires less space.